Completed Research Duplicate File Control Policy

Purpose

This document defines the duplicate-file control policy for the `Completed Research` repository.

The purpose of this policy is not to eliminate every repeated file in the repository. The purpose is to eliminate ambiguity.

A professional repository can contain multiple files with overlapping content when each file has a clearly defined role. Problems arise when multiple files appear equally valid and there is no immediate way to determine which one is the correct file to edit, reference, index, or trust.

This policy establishes a framework to:

- distinguish canonical files from backups, exports, archives, legacy files, templates, and prompt-generated artifacts
- reduce confusion during editing, review, and execution
- prevent drift between active and duplicated files
- support cleaner search, indexing, dashboarding, and validation
- preserve historical material without weakening operational clarity

This policy is designed to strengthen the repository that already exists. It is not intended to force unnecessary cleanup where duplication is already understandable and well-contained.

Core Principle

The goal is:

zero ambiguous duplicates

Not all duplicates are bad. Only unclear duplicates are bad.

A duplicate becomes a repository problem when one or more of the following are true:

- it is unclear which version is authoritative
- two files in active use contain overlapping but inconsistent information
- a backup is mistaken for a live file
- an export is mistaken for an editable source file
- an archived file remains too visible relative to the current file
- a prompt-generated derivative is not distinguished from a final operational document
- a user must open multiple files just to determine which one is current

The repository standard is not zero repetition.

The repository standard is zero confusion.

Policy Scope

This policy applies to files that overlap materially in purpose, content, or operational use.

This includes, but is not limited to:

- markdown source documents
- PDFs derived from markdown source files
- backup copies and dated snapshots
- archive copies of former active documents
- prompt-generated drafts or intermediate outputs
- duplicate checklists, trackers, or registers serving the same functional purpose
- mirrored files across active folders, checklist folders, export folders, or dashboard-related folders

This policy does not treat every repeated idea or every related file as a duplicate problem. It applies when overlap creates a realistic risk of confusion, drift, or incorrect use.

Core Operating Rule

For every meaningful operational purpose, there should be:

- one canonical editable source
- zero ambiguity about what should be edited
- clear classification for every overlapping file that is retained

Where multiple similar files are kept, each one must have an explicit role.

Duplicate File Role Categories

Every file that overlaps materially with another file should be classifiable into one of the following roles.

1. Canonical

The authoritative active file.

This is the file that should be edited, referenced, linked, indexed, and treated as the current source of truth.

2. Working Draft

An in-progress file that may later become canonical, but has not yet replaced the canonical file.

Working drafts are allowed temporarily, but should not remain unresolved indefinitely.

3. Backup

A preserved copy of a file created before a major edit, migration, refactor, rename, or restructuring pass.

Backups are valid and often useful, but they are not active source files.

4. Export

A derived output created for viewing, printing, sharing, or presentation.

Typical examples include PDFs, printable packets, presentation exports, and formatted deliverables.

Exports may repeat content from a canonical file, but must never be mistaken for the editable source.

5. Archive

A historical reference retained for recordkeeping, traceability, prior-state comparison, or rollback confidence.

Archived files are reference-only unless explicitly reinstated.

6. Legacy

A file preserved from a prior organizational system or naming pattern that is no longer part of the active standard.

Legacy files may still hold useful content, but they are not automatically current.

7. Prompt Artifact

A file created through prompting, generation, or intermediate automation that may have value as a working input or intermediate output, but is not automatically canonical.

8. Template

A reusable starting structure intended to be copied or instantiated, not edited as the live operational record.

Allowed Duplicates

The following duplicate relationships are acceptable when clearly labeled, properly located, and operationally understandable.

Allowed Case 1: canonical source + export

Example:

- `Operations_Strategy.md`
- `00_05_Operations_Run_Order_and_Standards.pdf`

This is allowed because the PDF is clearly a derived output and should not be mistaken for the editable source.

Allowed Case 2: canonical source + backup

Example:

- `08-REV__Pricing_Model.md`
- `08-REV__Pricing_Model__2026-04-05__v1.md`

This is allowed because the dated version is clearly a preserved state.

Allowed Case 3: canonical source + archived superseded version

Example:

- active file in an active section folder
- older version in that section's `.ARCHIVE/` or `98_Archive/`

This is allowed when the older version is clearly no longer live.

Allowed Case 4: canonical checklist + printable version

Example:

- editable markdown checklist
- dated PDF version prepared for review, printing, or binder use

This is allowed when the markdown file is the source and the PDF is clearly a derivative.

Allowed Case 5: template + instantiated operational file

Example:

- template in `91_Templates/`
- active version in an operational section

This is allowed because the files serve different roles.

Allowed Case 6: prompt-generated draft + reviewed canonical document

Example:

- draft in `.PROMPTS/` or a prompt-output staging area
- finalized operational file in the correct live section

This is allowed when the draft is not treated as automatically authoritative.

Not Allowed Duplicate Patterns

The following situations should be treated as structural problems and corrected.

Not Allowed Case 1: same active content in multiple active folders

If the same file content exists as co-equal live files in more than one active folder, the repository is vulnerable to confusion and drift.

Not Allowed Case 2: same checklist serving as editable source in multiple places

A checklist may be mirrored as an export or archive, but there should be one canonical editable source.

Not Allowed Case 3: backup files beside current files without clear labeling

If a user must open multiple files to discover which one is current, the duplicate policy has failed.

Not Allowed Case 4: archived files visually competing with active files

Archived files should not sit beside current files in a way that makes them appear equally active.

Not Allowed Case 5: prompt outputs indistinguishable from final deliverables

Generated content should not automatically inherit canonical status.

Not Allowed Case 6: duplicate filenames with only vague wording differences

Examples to avoid:

- `House_Rules.md`
- `House_Rules_Updated.md`
- `House_Rules_Final.md`
- `House_Rules_New_Final.md`

These create unnecessary ambiguity and should be resolved.

Not Allowed Case 7: exports becoming the only accessible version of important content

If the only clearly visible copy of an important operational document is a PDF or other export, the repository becomes harder to maintain safely.

Canonical Authority Rules

Every document with meaningful operational importance should have exactly one canonical source.

A canonical file should be the file that the repository expects a user to trust first.

Canonical source requirements

A canonical file should:

- live in the correct active folder
- be clearly distinguishable from backups, exports, archives, and prompt artifacts
- use the current repository naming standard when practical
- be listed in the section `FILE_REGISTER.md` when that control file exists
- be referenced by the relevant `INDEX.md` when that control file exists
- include YAML front matter when applicable
- be labeled with `authority: canonical`

Example YAML

```yaml
---
id: 05-OPS-MANUAL
title: Operations Manual
section: 05_Operations
doc_type: manual
status: active
authority: canonical
version: 1.0
```

```
created: 2026-04-05
updated: 2026-04-05
summary: Primary operating manual for the short-term rental.
---
```

Required Authority Labels in YAML

To reduce ambiguity, markdown documents that may overlap with other files should include an `authority` field.

Approved values

- `canonical`
- `working_draft`
- `backup`
- `export_source`
- `archived_reference`
- `legacy_reference`
- `template`
- `prompt_artifact`

Example: canonical

```
authority: canonical
```

Example: backup

```
authority: backup
```

Example: archived file

```
authority: archived_reference
```

Example: draft under review

```
authority: working_draft
```

This field should be treated as:

- mandatory for important active markdown documents

- mandatory when a markdown file materially overlaps another markdown file
- strongly recommended for archive-adjacent markdown files
- optional for small one-off notes that have no ambiguity risk

---

Folder Placement Rules

Duplicate control is not only about filenames. It is also about placement.

Correct placement reduces confusion before a file is even opened.

Canonical files belong in:

- active section folders
- section root when the file is intentionally a root control document
- `01_Core_Docs/`
- relevant tracker or structured operational folders

Backups belong in:

- `97_Backups/`
- section `98_Archive/` when retained locally for section history
- temporary local staging only during active work, not as a long-term pattern

Exports belong in:

- `96_Exports/`
- section `06_Exports/`
- printable binder or deliverable folders when clearly treated as derived output

Legacy files belong in:

- `99_Legacy/`
- clearly marked legacy subfolders if retained locally for section context

Archived files belong in:

- **Preferred:** `98_Archive/`
- **Acceptable:** `.ARCHIVE/`
- **Rule:** never use both in the same section

Prompt artifacts belong in:

- `92_Prompts/`
- prompt pack folders
- generation staging or intermediate folders if intentionally retained

Improper placement increases the chance that a valid duplicate becomes an ambiguous duplicate.

Repository-Specific Interpretation

This repository already contains multiple file-role types, including:

- active markdown source files
- PDFs
- checklists
- prompt packs
- validation reports
- archived materials
- dashboard or app-related files
- binder-ready outputs
- reference materials staged for future refinement

Because of that, duplicate control in this repository should focus on operational clarity, not on purging every repeated artifact.

The main question is not whether overlap exists.

The main question is whether a first-time reviewer could immediately tell:

- which file is current
- which file is editable
- which file is historical
- which file is generated
- which file is only for printing or sharing

If those distinctions are obvious, duplication may be acceptable.

If those distinctions are unclear, corrective action is required.

Duplicate Review Decision Framework

When duplicate files are found, classify each one using the following sequence.

Step 1: Determine whether the files are materially overlapping

Ask:

- do these files serve the same operational purpose
- would a user reasonably confuse one for the other
- do they contain overlapping source content
- are they both candidates for editing or reference

If no, they may not be duplicates in a meaningful sense.

If yes, continue.

Step 2: Assign a file role

Each file should be classified as:

- canonical

- working_draft
- backup
- export
- archive
- legacy
- prompt_artifact
- template

Step 3: Choose the action

Each duplicate should receive one of the following actions:

- keep as canonical
- keep as derivative
- move to backup
- move to archive
- move to legacy
- reclassify as export
- reclassify as prompt artifact
- merge into canonical
- rename for clarity
- delete

Step 4: Update repository references

If canonical status changes, update:

- `INDEX.md`
- `FILE_REGISTER.md`
- internal links
- dashboard references
- checklist references
- validation maps
- any known operational references pointing to the prior file

Step 5: Record the decision when the change is meaningful

Major duplicate resolutions should be documented in:

- a rename map
- a change log
- a validation report
- a duplicate control register

Duplicate Resolution Rules

Rule 1: One file must win

When two active files serve the same purpose, one must be designated canonical.

Rule 2: Historical value does not justify active duplication

If an older file is valuable for history, preserve it as an archive, not as a co-equal active file.

Rule 3: Exports should never replace editable source

A PDF, snapshot, or formatted deliverable should not become the only accessible version of important operational content.

Rule 4: Working drafts must be resolved

A draft can exist temporarily, but unresolved drafts should not accumulate without either promotion, archival, reclassification, or deletion.

Rule 5: Duplicate control must be recorded when it affects repository behavior

Major duplicate resolutions should be documented in a rename map, change log, validation report, or duplicate register.

Rule 6: Prompt-generated content must earn canonical status

Prompt-generated content may become useful operational content, but only after review, placement, and explicit promotion.

Rule 7: Root visibility matters

Even valid duplicate relationships become risky if folder-level visibility makes files look equally current.

Duplicate Audit Categories

When running a repository audit, duplicates should be reviewed by category.

Category A: Exact duplicates

Files that are identical or nearly identical.

Category B: Near duplicates

Files that overlap substantially but differ by partial edits, formatting, or incomplete updates.

Category C: Derivative duplicates

Files derived from the same source but intended for different roles, such as PDF exports or binder-ready outputs.

Category D: Structural duplicates

Files serving the same functional purpose in parallel systems.

Example:

- a master checklist in one folder
- a second editable checklist in another folder

These are often more dangerous than exact duplicates because they encourage parallel maintenance.

Category E: Promotion duplicates

Files that exist because a draft, prompt artifact, or migration candidate is in the process of becoming canonical.

These are acceptable temporarily, but must be resolved explicitly.

Duplicate Severity Levels

Use the following severity model during review.

Low severity

The duplicate is clearly labeled, properly located, and operationally understandable.

Example:

- canonical source plus dated PDF export

Medium severity

The duplicate has a legitimate purpose, but naming, placement, or labeling could still cause hesitation.

Example:

- backup stored too close to the live source

High severity

The duplicate could easily cause operational confusion.

Example:

- two active markdown files with overlapping content and no explicit authority

Critical severity

The duplicate causes direct governance failure.

Example:

- two different versions of the same operational manual both being used by the system

---

Recommended Duplicate Tracking File

Maintain a duplicate review register during major cleanup or restructuring passes.

Recommended filename

```
95-VALIDATION__Duplicate_Control_Register__YYYY-MM-DD.md
```

Recommended fields

- duplicate_group_id
- file_path
- file_role
- authority_status
- duplicate_type
- severity
- recommended_action
- action_owner
- action_status
- notes

This creates an auditable system rather than relying on memory.

Example Duplicate Control Decisions

Example 1: source + export

Decision:

- keep markdown as canonical
- keep PDF as export
- no issue if clearly named and located

Example 2: old operational manual beside new one

Decision:

- choose canonical version
- move old file to section archive
- rename archived file if necessary
- update `INDEX.md` and `FILE_REGISTER.md`

Example 3: two editable checklist files

Decision:

- identify which one is current
- merge missing information if needed
- retain one canonical source
- archive, reclassify, or delete the other

Example 4: prompt-generated draft of a policy

Decision:

- classify as `prompt_artifact` or `working_draft`
- do not let it replace the canonical file until reviewed and promoted

Example 5: root file and subfolder file appear to serve same role

Decision:

- determine which file is intended as the live operational source
- keep only one as canonical
- reclassify or relocate the other if retained

Promotion Rules

A duplicate-related file may only become canonical when:

- it has been reviewed
- it is complete enough for operational use
- it is placed in the correct active location
- internal references are updated as needed
- prior canonical status is retired, archived, or reclassified
- its YAML authority field is changed to `canonical`

Promotion should be explicit, not accidental.

A file does not become canonical merely because it is newer, more detailed, or prompt-generated.

It becomes canonical when the repository treats it as the current source of truth.

Legacy File Handling During Standardization

This policy accommodates the transition from pre-standard to post-standard repository practices.

Pre-Standard Files (Before 2026-04-06)

- may use legacy naming patterns
- may remain in legacy archive structures already in use
- should add `authority` field to YAML when identified as canonical
- should be renamed only when there is meaningful value in doing so
- should not trigger unnecessary cleanup if current role is already clear

Post-Standard Files (After 2026-04-06)

- should follow the current naming convention, subject to recognized exceptions
- must include `authority` field in YAML when applicable
- should not create new ambiguous duplicates
- should not rely on vague suffixes like `updated`, `final`, or `new`

Migration Priority

- **High Priority:** backups without clear dating, unclear file role labeling, or active duplicates with competing authority
- **Medium Priority:** canonical files missing `authority` YAML field
- **Low Priority:** files using recognized legacy naming patterns where role is already obvious

The standard should reduce confusion first.

Cosmetic normalization comes second.

Practical Rule of Thumb

When you see two similar files, ask one question first:

If someone opened this folder for the first time, would they immediately know which file to use, which file to edit, and which file to ignore?

If the answer is no, the duplicate policy is not being met.

Repository-Wide Best Practices

To prevent duplicate problems from recurring:

1. keep one canonical editable source per operational purpose
2. separate active, archive, backup, export, and prompt folders clearly
3. apply consistent naming conventions to new or promoted files
4. require authority labeling where overlap creates ambiguity
5. update section indexes and registers when files are promoted or retired
6. avoid storing backups beside live files long term
7. review prompt-generated outputs before allowing operational adoption
8. run duplicate audits during major restructuring, migration, or standardization phases
9. preserve useful history without allowing historical files to compete with live ones
10. treat placement, labeling, and visibility as part of duplicate control, not afterthoughts

---

Final Policy Statement

The `Completed Research` repository should preserve history without sacrificing clarity.

That means:

- one source of truth per operational purpose
- clear labeling for all overlapping files
- duplicate tolerance only when file roles are explicit
- archives, backups, exports, and prompt artifacts preserved in a controlled way
- no ambiguity about what should be edited, referenced, indexed, or trusted

The standard is not zero repetition.

The standard is zero confusion.# Completed Research Duplicate File Control Policy

Purpose

This document defines the duplicate-file control policy for the `Completed Research` repository.

The purpose of this policy is not to eliminate every repeated file in the repository. The purpose is to eliminate ambiguity.

A professional repository can contain multiple files with overlapping content when each file has a clearly defined role. Problems arise when multiple files appear equally valid and there is no immediate way to determine which one is the correct file to edit, reference, index, or trust.

This policy establishes a framework to:

- distinguish canonical files from backups, exports, archives, legacy files, templates, and prompt-generated artifacts
- reduce confusion during editing, review, and execution
- prevent drift between active and duplicated files
- support cleaner search, indexing, dashboarding, and validation
- preserve historical material without weakening operational clarity

This policy is designed to strengthen the repository that already exists. It is not intended to force unnecessary cleanup where duplication is already understandable and well-contained.

Core Principle

The goal is:

zero ambiguous duplicates

Not all duplicates are bad. Only unclear duplicates are bad.

A duplicate becomes a repository problem when one or more of the following are true:

- it is unclear which version is authoritative
- two files in active use contain overlapping but inconsistent information
- a backup is mistaken for a live file
- an export is mistaken for an editable source file
- an archived file remains too visible relative to the current file
- a prompt-generated derivative is not distinguished from a final operational document
- a user must open multiple files just to determine which one is current

The repository standard is not zero repetition.

The repository standard is zero confusion.

Policy Scope

This policy applies to files that overlap materially in purpose, content, or operational use.

This includes, but is not limited to:

- markdown source documents
- PDFs derived from markdown source files
- backup copies and dated snapshots
- archive copies of former active documents
- prompt-generated drafts or intermediate outputs
- duplicate checklists, trackers, or registers serving the same functional purpose
- mirrored files across active folders, checklist folders, export folders, or dashboard-related folders

This policy does not treat every repeated idea or every related file as a duplicate problem. It applies when overlap creates a realistic risk of confusion, drift, or incorrect use.

Core Operating Rule

For every meaningful operational purpose, there should be:

- one canonical editable source
- zero ambiguity about what should be edited
- clear classification for every overlapping file that is retained

Where multiple similar files are kept, each one must have an explicit role.

Duplicate File Role Categories

Every file that overlaps materially with another file should be classifiable into one of the following roles.

1. Canonical

The authoritative active file.

This is the file that should be edited, referenced, linked, indexed, and treated as the current source of truth.

2. Working Draft

An in-progress file that may later become canonical, but has not yet replaced the canonical file.

Working drafts are allowed temporarily, but should not remain unresolved indefinitely.

3. Backup

A preserved copy of a file created before a major edit, migration, refactor, rename, or restructuring pass.

Backups are valid and often useful, but they are not active source files.

4. Export

A derived output created for viewing, printing, sharing, or presentation.

Typical examples include PDFs, printable packets, presentation exports, and formatted deliverables.

Exports may repeat content from a canonical file, but must never be mistaken for the editable source.

5. Archive

A historical reference retained for recordkeeping, traceability, prior-state comparison, or rollback confidence.

Archived files are reference-only unless explicitly reinstated.

6. Legacy

A file preserved from a prior organizational system or naming pattern that is no longer part of the active standard.

Legacy files may still hold useful content, but they are not automatically current.

7. Prompt Artifact

A file created through prompting, generation, or intermediate automation that may have value as a working input or intermediate output, but is not automatically canonical.

8. Template

A reusable starting structure intended to be copied or instantiated, not edited as the live operational record.

Allowed Duplicates

The following duplicate relationships are acceptable when clearly labeled, properly located, and operationally understandable.

Allowed Case 1: canonical source + export

Example:

- `Operations_Strategy.md`
- `00_05_Operations_Run_Order_and_Standards.pdf`

This is allowed because the PDF is clearly a derived output and should not be mistaken for the editable source.

Allowed Case 2: canonical source + backup

Example:

- `08-REV__Pricing_Model.md`
- `08-REV__Pricing_Model__2026-04-05__v1.md`

This is allowed because the dated version is clearly a preserved state.

Allowed Case 3: canonical source + archived superseded version

Example:

- active file in an active section folder
- older version in that section's `.ARCHIVE/` or `98_Archive/`

This is allowed when the older version is clearly no longer live.

Allowed Case 4: canonical checklist + printable version

Example:

- editable markdown checklist
- dated PDF version prepared for review, printing, or binder use

This is allowed when the markdown file is the source and the PDF is clearly a derivative.

Allowed Case 5: template + instantiated operational file

Example:

- template in `91_Templates/`
- active version in an operational section

This is allowed because the files serve different roles.

Allowed Case 6: prompt-generated draft + reviewed canonical document

Example:

- draft in `.PROMPTS/` or a prompt-output staging area
- finalized operational file in the correct live section

This is allowed when the draft is not treated as automatically authoritative.

Not Allowed Duplicate Patterns

The following situations should be treated as structural problems and corrected.

Not Allowed Case 1: same active content in multiple active folders

If the same file content exists as co-equal live files in more than one active folder, the repository is vulnerable to confusion and drift.

Not Allowed Case 2: same checklist serving as editable source in multiple places

A checklist may be mirrored as an export or archive, but there should be one canonical editable source.

Not Allowed Case 3: backup files beside current files without clear labeling

If a user must open multiple files to discover which one is current, the duplicate policy has failed.

Not Allowed Case 4: archived files visually competing with active files

Archived files should not sit beside current files in a way that makes them appear equally active.

Not Allowed Case 5: prompt outputs indistinguishable from final deliverables

Generated content should not automatically inherit canonical status.

Not Allowed Case 6: duplicate filenames with only vague wording differences

Examples to avoid:

- `House_Rules.md`
- `House_Rules_Updated.md`
- `House_Rules_Final.md`
- `House_Rules_New_Final.md`

These create unnecessary ambiguity and should be resolved.

Not Allowed Case 7: exports becoming the only accessible version of important content

If the only clearly visible copy of an important operational document is a PDF or other export, the repository becomes harder to maintain safely.

Canonical Authority Rules

Every document with meaningful operational importance should have exactly one canonical source.

A canonical file should be the file that the repository expects a user to trust first.

Canonical source requirements

A canonical file should:

- live in the correct active folder
- be clearly distinguishable from backups, exports, archives, and prompt artifacts
- use the current repository naming standard when practical
- be listed in the section `FILE_REGISTER.md` when that control file exists
- be referenced by the relevant `INDEX.md` when that control file exists
- include YAML front matter when applicable
- be labeled with `authority: canonical`

Example YAML

```yaml
---
id: 05-OPS-MANUAL
title: Operations Manual
section: 05_Operations
doc_type: manual
status: active
authority: canonical
version: 1.0
created: 2026-04-05
updated: 2026-04-05
```

```
summary: Primary operating manual for the short-term rental.
---
```

Required Authority Labels in YAML

To reduce ambiguity, markdown documents that may overlap with other files should include an `authority` field.

Approved values

- `canonical`
- `working_draft`
- `backup`
- `export_source`
- `archived_reference`
- `legacy_reference`
- `template`
- `prompt_artifact`

Example: canonical

```
authority: canonical
```

Example: backup

```
authority: backup
```

Example: archived file

```
authority: archived_reference
```

Example: draft under review

```
authority: working_draft
```

This field should be treated as:

- mandatory for important active markdown documents
- mandatory when a markdown file materially overlaps another markdown file
- strongly recommended for archive-adjacent markdown files

- optional for small one-off notes that have no ambiguity risk

---

Folder Placement Rules

Duplicate control is not only about filenames. It is also about placement.

Correct placement reduces confusion before a file is even opened.

Canonical files belong in:

- active section folders
- section root when the file is intentionally a root control document
- `01_Core_Docs/`
- relevant tracker or structured operational folders

Backups belong in:

- `97_Backups/`
- section `98_Archive/` when retained locally for section history
- temporary local staging only during active work, not as a long-term pattern

Exports belong in:

- `96_Exports/`
- section `06_Exports/`
- printable binder or deliverable folders when clearly treated as derived output

Legacy files belong in:

- `99_Legacy/`
- clearly marked legacy subfolders if retained locally for section context

Archived files belong in:

- **Preferred:** `98_Archive/`
- **Acceptable:** `.ARCHIVE/`
- **Rule:** never use both in the same section

Prompt artifacts belong in:

- `92_Prompts/`
- prompt pack folders
- generation staging or intermediate folders if intentionally retained

Improper placement increases the chance that a valid duplicate becomes an ambiguous duplicate.

Repository-Specific Interpretation

This repository already contains multiple file-role types, including:

- active markdown source files
- PDFs
- checklists
- prompt packs
- validation reports
- archived materials
- dashboard or app-related files
- binder-ready outputs
- reference materials staged for future refinement

Because of that, duplicate control in this repository should focus on operational clarity, not on purging every repeated artifact.

The main question is not whether overlap exists.

The main question is whether a first-time reviewer could immediately tell:

- which file is current
- which file is editable
- which file is historical
- which file is generated
- which file is only for printing or sharing

If those distinctions are obvious, duplication may be acceptable.

If those distinctions are unclear, corrective action is required.

Duplicate Review Decision Framework

When duplicate files are found, classify each one using the following sequence.

Step 1: Determine whether the files are materially overlapping

Ask:

- do these files serve the same operational purpose
- would a user reasonably confuse one for the other
- do they contain overlapping source content
- are they both candidates for editing or reference

If no, they may not be duplicates in a meaningful sense.

If yes, continue.

Step 2: Assign a file role

Each file should be classified as:

- canonical
- working_draft
- backup

- export
- archive
- legacy
- prompt_artifact
- template

Step 3: Choose the action

Each duplicate should receive one of the following actions:

- keep as canonical
- keep as derivative
- move to backup
- move to archive
- move to legacy
- reclassify as export
- reclassify as prompt artifact
- merge into canonical
- rename for clarity
- delete

Step 4: Update repository references

If canonical status changes, update:

- `INDEX.md`
- `FILE_REGISTER.md`
- internal links
- dashboard references
- checklist references
- validation maps
- any known operational references pointing to the prior file

Step 5: Record the decision when the change is meaningful

Major duplicate resolutions should be documented in:

- a rename map
- a change log
- a validation report
- a duplicate control register

Duplicate Resolution Rules

Rule 1: One file must win

When two active files serve the same purpose, one must be designated canonical.

Rule 2: Historical value does not justify active duplication

If an older file is valuable for history, preserve it as an archive, not as a co-equal active file.

Rule 3: Exports should never replace editable source

A PDF, snapshot, or formatted deliverable should not become the only accessible version of important operational content.

Rule 4: Working drafts must be resolved

A draft can exist temporarily, but unresolved drafts should not accumulate without either promotion, archival, reclassification, or deletion.

Rule 5: Duplicate control must be recorded when it affects repository behavior

Major duplicate resolutions should be documented in a rename map, change log, validation report, or duplicate register.

Rule 6: Prompt-generated content must earn canonical status

Prompt-generated content may become useful operational content, but only after review, placement, and explicit promotion.

Rule 7: Root visibility matters

Even valid duplicate relationships become risky if folder-level visibility makes files look equally current.

Duplicate Audit Categories

When running a repository audit, duplicates should be reviewed by category.

Category A: Exact duplicates

Files that are identical or nearly identical.

Category B: Near duplicates

Files that overlap substantially but differ by partial edits, formatting, or incomplete updates.

Category C: Derivative duplicates

Files derived from the same source but intended for different roles, such as PDF exports or binder-ready outputs.

Category D: Structural duplicates

Files serving the same functional purpose in parallel systems.

Example:

- a master checklist in one folder
- a second editable checklist in another folder

These are often more dangerous than exact duplicates because they encourage parallel maintenance.

Category E: Promotion duplicates

Files that exist because a draft, prompt artifact, or migration candidate is in the process of becoming canonical.

These are acceptable temporarily, but must be resolved explicitly.

Duplicate Severity Levels

Use the following severity model during review.

Low severity

The duplicate is clearly labeled, properly located, and operationally understandable.

Example:

- canonical source plus dated PDF export

Medium severity

The duplicate has a legitimate purpose, but naming, placement, or labeling could still cause hesitation.

Example:

- backup stored too close to the live source

High severity

The duplicate could easily cause operational confusion.

Example:

- two active markdown files with overlapping content and no explicit authority

Critical severity

The duplicate causes direct governance failure.

Example:

- two different versions of the same operational manual both being used by the system

---

Recommended Duplicate Tracking File

Maintain a duplicate review register during major cleanup or restructuring passes.

Recommended filename

```
95-VALIDATION__Duplicate_Control_Register__YYYY-MM-DD.md
```

Recommended fields

- duplicate_group_id
- file_path
- file_role
- authority_status
- duplicate_type
- severity
- recommended_action
- action_owner
- action_status
- notes

This creates an auditable system rather than relying on memory.

Example Duplicate Control Decisions

Example 1: source + export

Decision:

- keep markdown as canonical
- keep PDF as export
- no issue if clearly named and located

Example 2: old operational manual beside new one

Decision:

- choose canonical version
- move old file to section archive
- rename archived file if necessary
- update `INDEX.md` and `FILE_REGISTER.md`

Example 3: two editable checklist files

Decision:

- identify which one is current
- merge missing information if needed
- retain one canonical source
- archive, reclassify, or delete the other

Example 4: prompt-generated draft of a policy

Decision:

- classify as `prompt_artifact` or `working_draft`
- do not let it replace the canonical file until reviewed and promoted

Example 5: root file and subfolder file appear to serve same role

Decision:

- determine which file is intended as the live operational source
- keep only one as canonical
- reclassify or relocate the other if retained

Promotion Rules

A duplicate-related file may only become canonical when:

- it has been reviewed
- it is complete enough for operational use
- it is placed in the correct active location
- internal references are updated as needed
- prior canonical status is retired, archived, or reclassified
- its YAML authority field is changed to `canonical`

Promotion should be explicit, not accidental.

A file does not become canonical merely because it is newer, more detailed, or prompt-generated.

It becomes canonical when the repository treats it as the current source of truth.

Legacy File Handling During Standardization

This policy accommodates the transition from pre-standard to post-standard repository practices.

Pre-Standard Files (Before 2026-04-06)

- may use legacy naming patterns
- may remain in legacy archive structures already in use
- should add `authority` field to YAML when identified as canonical
- should be renamed only when there is meaningful value in doing so
- should not trigger unnecessary cleanup if current role is already clear

Post-Standard Files (After 2026-04-06)

- should follow the current naming convention, subject to recognized exceptions
- must include `authority` field in YAML when applicable
- should not create new ambiguous duplicates
- should not rely on vague suffixes like `updated`, `final`, or `new`

Migration Priority

- **High Priority:** backups without clear dating, unclear file role labeling, or active duplicates with competing authority
- **Medium Priority:** canonical files missing `authority` YAML field
- **Low Priority:** files using recognized legacy naming patterns where role is already obvious

The standard should reduce confusion first.

Cosmetic normalization comes second.

Practical Rule of Thumb

When you see two similar files, ask one question first:

If someone opened this folder for the first time, would they immediately know which file to use, which file to edit, and which file to ignore?

If the answer is no, the duplicate policy is not being met.

Repository-Wide Best Practices

To prevent duplicate problems from recurring:

1. keep one canonical editable source per operational purpose
2. separate active, archive, backup, export, and prompt folders clearly
3. apply consistent naming conventions to new or promoted files
4. require authority labeling where overlap creates ambiguity
5. update section indexes and registers when files are promoted or retired
6. avoid storing backups beside live files long term
7. review prompt-generated outputs before allowing operational adoption
8. run duplicate audits during major restructuring, migration, or standardization phases
9. preserve useful history without allowing historical files to compete with live ones
10. treat placement, labeling, and visibility as part of duplicate control, not afterthoughts

---

Final Policy Statement

The `Completed Research` repository should preserve history without sacrificing clarity.

That means:

- one source of truth per operational purpose
- clear labeling for all overlapping files
- duplicate tolerance only when file roles are explicit
- archives, backups, exports, and prompt artifacts preserved in a controlled way
- no ambiguity about what should be edited, referenced, indexed, or trusted

The standard is not zero repetition.

The standard is zero confusion.

Professional Review Addendum (2026-04-06)

The following addendum merges professional governance improvements into this standard while preserving all original authored content above.

Added Governance Clarifications

Completed Research Duplicate File Control Policy (v2)

Purpose

Define duplicate governance for `Completed Research` with one objective: **zero ambiguous duplicates**.

This policy allows intentional overlap (exports, archives, backups, prompts) when role and authority are explicit.

Scope

Applies to files that overlap in purpose/content, especially:

- markdown source docs
- backups/snapshots
- exports (PDF, printable outputs)
- archive/legacy files
- prompt-generated artifacts

Precedence

If standards conflict, apply in this order:

1. Folder structure blueprint (placement and section control surfaces)
2. This duplicate policy (authority and overlap decisions)
3. Naming convention (file naming patterns)
4. YAML standard (metadata completeness)

Core Rule

For each operational purpose:

- one canonical editable source
- clear role classification for retained overlaps
- no uncertainty about what to edit/trust

File Roles

Approved roles:

- `canonical`
- `working_draft`
- `backup`

- `export_source`
- `archived_reference`
- `legacy_reference`
- `template`
- `prompt_artifact`

Allowed Duplicate Patterns

Allowed when explicit and well-placed:

- canonical markdown + dated export PDF
- canonical + dated backup snapshot
- canonical + archived superseded version
- template + instantiated operational file
- prompt artifact + reviewed canonical promotion target

Not Allowed Patterns

- Co-equal active files for same purpose
- Backup files that look like live sources
- Export-only visibility where source is missing/unclear
- Parallel editable checklists serving same purpose
- Vague suffix collisions (`final/new/updated`) in active canonical sets

Placement Rules

Compatibility model for this repository:

- Preferred utility naming: `90_`–`99_` folders
- Accepted current aliases: `.INBOX`, `.PROMPTS`, `.CHECKLISTS`, `.ARCHIVE`, `.APP`

Canonical placement by role:

- backups: `97_Backups/` (or section archive when local historical context is needed)
- exports: `96_Exports/` or section `06_Exports/`
- archives: `98_Archive/` or `.ARCHIVE/` (not both in same section)
- prompt artifacts: `92_Prompts/` or `.PROMPTS/`
- legacy: `99_Legacy/`

Decision Workflow

1. Confirm material overlap.
2. Assign each file a role.
3. Choose action: keep, merge, reclassify, archive, rename, or remove.
4. Update references (`INDEX.md`, `FILE_REGISTER.md`, links) if canonical changes.
5. Record material decisions in validation/change logs.

Severity Model

- Low: clear role + clear location

- Medium: valid overlap but weak signaling
- High: likely operational confusion
- Critical: competing active authority

Promotion Rule

A non-canonical file becomes canonical only when:

- reviewed and approved for operational use
- placed in active location
- references updated
- prior canonical is retired/reclassified
- YAML `authority` updated to `canonical`

Transition Policy

- Pre-standard files may keep legacy naming/location if role is already clear.
- Post-standard changes must avoid creating new ambiguity.
- Prioritize high-risk ambiguity first, cosmetic normalization later.

Final Standard

The repository can preserve history and generated artifacts without confusion if:

- one source of truth exists per operational purpose
- overlap roles are explicit
- placement and authority are visible at a glance